

Projekat iz predmeta Digitalni VLSI Sistemi

Ubrzanje filtriranja slike

Vasilije Čarapić 2022/0218

Nikola Ristanović 2022/0469

Beograd, 29. jul 2026.

Sadržaj

1	Uvod	2
2	Arhitektura HW/SW Sistema	2
2.1	Adresna Mapa i Kontrolni Registri (<code>acc_ctrl.h</code>)	2
3	Softverska arhitektura i opis API-ja	4
3.1	Sistemska kontrola i uslužne funkcije	4
3.2	Softverski filter	4
3.3	Upravljanje hardverskim akceleratorom i verifikacija	5
3.4	Sinteza i konfiguracija koeficijenata filtera	5
3.5	Drajver AXI DMA kontrolera	5
3.6	API za konfiguraciju registara akceleratora	6
4	Opis VHDL blokova i njihova funkcionalnost kroz portove	6
4.1	Register File (<code>register_file</code>)	6
4.2	Params Loader (<code>params_loader</code>)	7
4.3	Address Generator (<code>address_gen</code>)	7
4.4	Convolution Core (<code>convolution_core</code>)	7
4.5	Output Controller (<code>output_controller</code>)	8
4.6	ALU Logic (<code>alu_logic</code>)	8
4.7	Filter Config (<code>filter_config</code>)	8
5	Rezultati	9
5.1	Primeri slika rezultata	10
6	Zaključak	10

Sažetak

U ovom izveštaju prikazano je rešenje za ubrzanje prostornog filtriranja digitalnih slika na Zynq-7000 SoC platformi. Sistem kombinuje softversku aplikaciju na ARM Cortex-A9 procesoru i hardverski akcelerator (`acc_image_filter`) u FPGA logici koji podržava prozor konvolucije do dimenzija 9×9 ($R_{max} = 4$). Konfigurisanje akceleratora vrši se preko AXI-Lite registra sa specifičnim adresnim preslikavanjem (`acc_ctrl.h`), dok je prenos slika realizovan preko AXI DMA kontrolera sa AXI Stream interfejsom u prekidnom režimu rada (`dma_utils`). Podržano je dinamičko izračunavanje koeficijenata u Q12 formatu fiksne tačke za različite tipove filtara (Box, Gauss, LoG, Sharpen, Sobel). Verifikacija je izvršena poređenjem izlaza akceleratora sa softverskim referentnim modelom (`imageFilterSW`) uz provere konzistentnosti keš memorije.

1 Uvod

Prostorno filtriranje slika (konvolucija) zahteva obradu velikog broja piksela u realnom vremenu, što značajno opterećuje procesor. Hibridni SoC sistemi poput Xilinx Zynq-7000 omogućavaju rasterećenje procesora prebacivanjem računski intenzivnih operacija na namenske akceleratoru u FPGA strukturi.

Cilj ovog projekta je implementacija i verifikacija HW/SW sistema za filtriranje slika sa podrškom za više vrsta filtara, dinamičku alokaciju memorije i konfiguraciju u realnom vremenu.

2 Arhitektura HW/SW Sistema

Sistem obuhvata sledeće gradivne elemente:

- **Processing System (PS):** ARM Cortex-A9 procesor koji vodi proces testiranja, izračunava referentni SW rezultat (`imageFilterSW`) i konfiguriše parametre obrade.
- **Programmable Logic (PL):**
 - **acc_image_filter:** Hardverski akcelerator za konvoluciju slike sa prozorom obrade do 9×9 piksela (maksimalno 81 koeficijent).
 - **AXI DMA:** Kontroler za direktan prenos piksela iz DRAM memorije ka akceleratoru (TX) i prijem filtrirane slike (RX).
 - **AXI Timer (XTmrCtr):** Hardware tajmer za merenje ciklusa i vremena izvršavanja u mikrosekundama (μs).

2.1 Adresna Mapa i Kontrolni Registri (`acc_ctrl.h`)

Hardverski akcelerator se konfiguriše upisom u registre preko AXI-Lite magistrale. S obzirom na to da VHDL interna logika vrši šifrovanje adrese kao `reg_waddr = awaddr >> 2` uz korak od 2, ARM procesor mora generisati adrese pomaknute za faktor 4 (`ARM_adresa = VHDL_indeks  $\times$  4`).

Struktura kontrolnog registra na adresi 0x00 organizovana je po bitima:

Tabela 1: Adresna mapa kontrolnih registra akceleratora.

Registari	ARM Adresa	Opis / Funkcija
REG_CTRL_ADDR	0x00	Kontrolni bitovi (Mode, Bypass, Border, BorderVal)
REG_RADIUS_ADDR	0x08	Radijus konvolucionog filtra ($0 \leq R \leq 4$)
REG_IMG_W_ADDR	0x10	Širina ulazne slike (<i>imgW</i>)
REG_IMG_H_ADDR	0x18	Visina ulazne slike (<i>imgH</i>)
REG_COEFF_SCALE_ADDR	0x20	Skalarni faktor koeficijenata (Q12 format)
REG_COEFF_BASE_ADDR	0x28	Bazna adresa koeficijenata konvolucije
COEFF_ADDR(<i>i</i>)	0x28 + <i>i</i> *8	Adresa <i>i</i> -tog koeficijenta ($0 \leq i < 81$)

- CTRL_MODE_BIT (bit 0): Izbor režima rada (Mode 0 ili Mode 1).
- CTRL_BYPASS_BIT (bit 1): Aktiviranje direktnog prolaza bez obrade.
- CTRL_BORD_SHIFT (bit 2): Režim obrade ivica (Crop, Nearest, Constant).
- CTRL_BVAL_SHIFT (bit 4): Konstantna vrednost piksela za vanjske ivice.

3 Softverska arhitektura i opis API-ja

Softverski subsistem koji se izvršava na ARM Cortex-A9 procesoru (PS) odgovoran je za inicijalizaciju sistema, alokaciju memorije, sintezu koeficijenata filtera, izračunavanje softverske referentne vrednosti, kontrolu AXI DMA prenosa i konfiguraciju hardvera. Softverska arhitektura je modularizovana u nekoliko namenskih zaglavlja (header-a) i izvornih komponenti.

3.1 Sistemska kontrola i uslužne funkcije

Glavna logika za orkestraciju u fajlu `main.c` upravlja rasporedom slike u memoriji, izračunava dimenzije frejma i rukuje raspakivanjem izlaza.

- `static void calcOutputDims(u16* outW, u16* outH, ProcessingParams* params);`
Izračunava prostorne dimenzije obrađene izlazne slike na osnovu širine i visine ulaznog frejma, radijusa filtera R i režima obrade ivica (crop, nearest ili constant).

3.2 Softverski filter

Softverski filter obezbeđuje referentnu implementaciju radi provere tačnosti i performansi izvršavanja PL akceleratora.

- `void imageFilterSW(u8* DataIn, u16* DataOut, ProcessingParams* Params);`
Vrši 2D prostornu konvoluciju u potpunosti u softveru nad baferom ulazne slike `DataIn` koristeći parametre definisane u `Params`, i skladišti rezultat u `DataOut`.

3.3 Upravljanje hardverskim akceleratorom i verifikacija

Funkcije definisane u `hw_image_filter.h` premošćuju jaz između upravljačkog (drajverskog) sloja softvera, izvršavanja u hardveru i verifikacije.

- `int imageFilterHW(u8 *DataIn, u16 *DataOut, ProcessingParams Params);`
Konfiguriše registre akceleratora, podešava AXI DMA deskriptore, pokreće striming prenose i čeka na završetak izvršavanja.
- `int CheckData(s16 *DataOut, u16 *ReferentBuffer, ProcessingParams Params, u32 OutputSize);`
Poređi izlazni bafer obrađen u hardveru sa softverskim referentnim baferom element po element, nakon što invalidira keš podatke CPU-a.
- `void calcOutputSize(u32 *outputSize, ProcessingParams *params);`
Izračunava ukupne zahteve za alokacijom memorije piksela za izlazni bafer na osnovu izabranog režima rukovanja ivicama i radijusa prozora.

3.4 Sinteza i konfiguracija koeficijenata filtera

Moduli za generisanje jezgra (kernel-a) filtera sintetišu koeficijente prostornog filtera u zapisu sa fiksno zapetom.

- `void generateCoefficients(ProcessingParams* params, filt_type_t FILTER_TYPE, float sigma, float k, float inv_scale);`
Generiše dinamičke koeficijente prostornog filtera za tipove koji uključuju Box, Gauss, Laplacian of Gaussian, Sharpening i Sobel jezgra na osnovu standardne devijacije σ i parametara skaliranja.
- `void printCoeffs(int16_t s16_arr[MAX_WIN * MAX_WIN]);`
Uslužna funkcija za otklanjanje grešaka (debug) koja ispisuje sintetišane koeficijente 2D niza na terminal.
- `void setFilterConfig(ProcessingParams* Params, FilterConfig config);`
Primenjuje unapred definisane statičke konfiguracije filtera (Enum `FilterConfig`) na strukturu parametara.

3.5 Drajver AXI DMA kontrolera

Interfejs za DMA apstrakciju upravlja neblokirajućim prenosom podataka u vidu strim-a između DDR memorije i hardverskog akceleratora.

- `int DmaConfigure(XAxiDma_Config *AxiDmaConfigPtr, XAxiDma *AxiDmaPtr);`
Inicijalizuje instancu AXI DMA motora i registruje rukovaoce prekida za smerove memory-to-stream (MM2S) i stream-to-memory (S2MM).
- `int DmaStartTransfers(XAxiDma *AxiDmaPtr, u8 *TxBuffer, u32 TxSize, u16 *RxBuffer, u32 RxSize);`
Pokreće asinkrone DMA prenose tako što prvo podešava S2MM (prijemni) kanal kako bi se sprečio gubitak paketa, nakon čega sledi okidanje MM2S (predajnog) kanala.



Slika 5: Hierarhijska struktura hardvera

- `int DmaWaitTransfers(volatile u32 *TxFlag, volatile u32 *RxFlag, u32 Timeout);`
Proverava flegove statusa prenosa (polling) uz unapred definisan tajmout iteracije kako bi se osiguralo da su i TX i RX operacije bezbedno završene.

3.6 API za konfiguraciju registara akceleratora

Nisko-nivoarska konfiguracija registara preslikanih u memorijski prostor (memory-mapped) za akcelerski IP u FPGA-u vrši se preko `acc_ctrl.h`.

- `int AccConfigure(UINTPTR BaseAddress, ProcessingParams Params);`
Upisuje parametre izvršavanja (Mode, Bypass, Border podešavanja, Radius, Width, Height, Coefficient Scale i 81 vrednost koeficijenata) u prilagođeni AXI-Lite registerski prostor IP jezgra.

4 Opis VHDL blokova i njihova funkcionalnost kroz portove

U ovom poglavlju dat je sažet pregled ključnih VHDL modulâ koji čine hardversku arhitekturu akceleratora za obradu slika, sa fokusom na njihovu funkcionalnost i ulazno-izlazne portove. Glavna komponenta je *acc_image_filter* koja portovima objedinjuje sve deleve sistema.

Za potrebe projekta, takođe je urađen bonus 2.

4.1 Register File (`register_file`)

- **Funkcionalnost:** Prihvatanje, skladištenje i čitanje konfiguracionih parametara akceleratora (dimenzije slike, radijus, koeficijenti filtra) putem AXI4-Lite interfejsa.
- **Ključni portovi:**

- `clk`, `reset`: Takt i sinhroni reset.
- `reg_waddr`, `wdata`, `wstrb`, `axi_write_ready`: AXI upisni interfejs.
- `lock_config`: Blokira izmjenu registara tokom obrade slike.
- `reg_raddr`, `rdata`: AXI čitaći interfejs.
- `reg_ctrl_out`, `reg_radius_out`, `reg_img_w_out`, `reg_img_h_out`, `reg_coeffs_out`: Izlazni konfiguracioni parametri proslijeđeni ostalim modulima.
- `regs_written`: Statusni signal koji označava završen upis svih registara.

4.2 Params Loader (`params_loader`)

- **Funkcionalnost:** Dekodiranje sirovih parametara iz registarskog fajla u stanju konfiguracije (`Config`) i priprema radnih parametara za filtriranje (proračun ivica, modova i skaliranja).
- **Ključni portovi:**
 - `reg_ctrl`, `reg_radius`, `reg_img_w`, `reg_img_h`, `filter_coeffs`: Sirovi konfiguracioni ulazi.
 - `state`: Ulazno stanje FSM-a za kontrolu trenutka prihvatanja parametara.
 - `imgW`, `imgH`: Proračunate izlazne dimenzije slike (uzimajući u obzir *padding*).
 - `img_r`, `div_shift`, `mode`, `bypass`: Kodirani parametri radijusa, diferencijalnog pomjeraja, formata podataka (8-bit neoznačeni / 16-bit fiksni zarez) i zaobljenja obrade.
 - `nearest_en`, `border_en`: Kontrolni signali za obradu rubnih piksela.

4.3 Address Generator (`address_gen`)

- **Funkcionalnost:** Generisanje adresa unutrašnjih BRAM linijskih bafera za formiranje klizajućeg prozora obrade piksela.
- **Ključni portovi:**
 - `ram_en`, `img_processing`, `skip_mode`: Upravljanje i zamrzavanje adresiranja u slučaju zastoja protoka (*backpressure*).
 - `row`, `col`, `img_r`, `shift`, `imgW`: Geometrijski parametri trenutnog piksela i slike.
 - `addra`, `addrb`, `enb`, `wea`: Adresni i kontrolni signali za BRAM memorijske portove A i B.
 - `rd_modulo`, `wr_modulo`, `data_modulo`: Indeksi za kružno (modulo) adresiranje unutar bafera.

4.4 Convolution Core (`convolution_core`)

- **Funkcionalnost:** Realizacija protočne (*pipelined*) 2D konvolucije – paralelno množenje matrice piksela (9×9) sa koeficijentima i stablasto sabiranje sa zasićenjem.
- **Ključni portovi:**

- `pipe_en`: Signal dozvole pajplajna (zamrzava rad pri *backpressure*-u).
- `alu`, `filter_coeffs`, `reg_coeff_scale`: Ulazna matrica piksela (81 piksel), koeficijenti filtera i faktor skaliranja.
- `mode`: Određuje da li je izlaz 8-bitni piksel ili 16-bitna vrednost sa fiksnim zarezom.
- `acc_out`: Izlazna 16-bitna akumulirana, skalirana i zasićena vrednost obrađenog piksela.

4.5 Output Controller (`output_controller`)

- **Funkcionalnost:** Sinhronizacija i pakovanje izlaznih piksela, preusmeravanje podataka (obraditi / *bypass*) i generisanje AXI-Stream kontrolnih signala.
- **Ključni portovi:**
 - `input_pixel`, `acc_out`: Ulazni piksel (za *bypass*) i filtrirani piksel iz konvolucionog jezgra.
 - `pipe_en`, `in_handshake`, `mode`, `img_processing`, `skip_mode`: Kontrolni signali protoka i odabira režima.
 - `output_val`, `output_enable`, `last_pixel`: Formatirani izlazni podaci, `tvalid` i `tlast` signali interfejsa.
 - `processing_done`: Signal završetka obrade kompletnog okvira.

4.6 ALU Logic (`alu_logic`)

- **Funkcionalnost:** Obuhvatni modul koji povezuje `address_gen`, `window_engine`, `convolution_core` i `output_controller` u integrisanu cjelinu za obradu.
- **Ključni portovi:**
 - `input_pixel`, `reg_tvalid`, `reg_tlast`: Ulazni strujni podaci sa pratećim sinhronizacionim signalima.
 - `img_r`, `mode`, `imgW`, `imgH`, `shift`, `filter_coeffs`, `reg_coeff_scale`: Konfiguracioni ulazi proslijeđeni unutrašnjim blokovima.
 - `output_enable`, `last_pixel`, `processing_done`, `pixel_output`: Konačni izlazi modula ka vanjskom sistemu.

4.7 Filter Config (`filter_config`)

- **Funkcionalnost:** VHDL paket (`package`) koji definiše tipove podataka, konstante, LUT tabele za radijus i skaliranje, te matrice koeficijenata za unapred definisane filtre (Box, Gaussian, LOG, Sharpen).

5 Rezultati

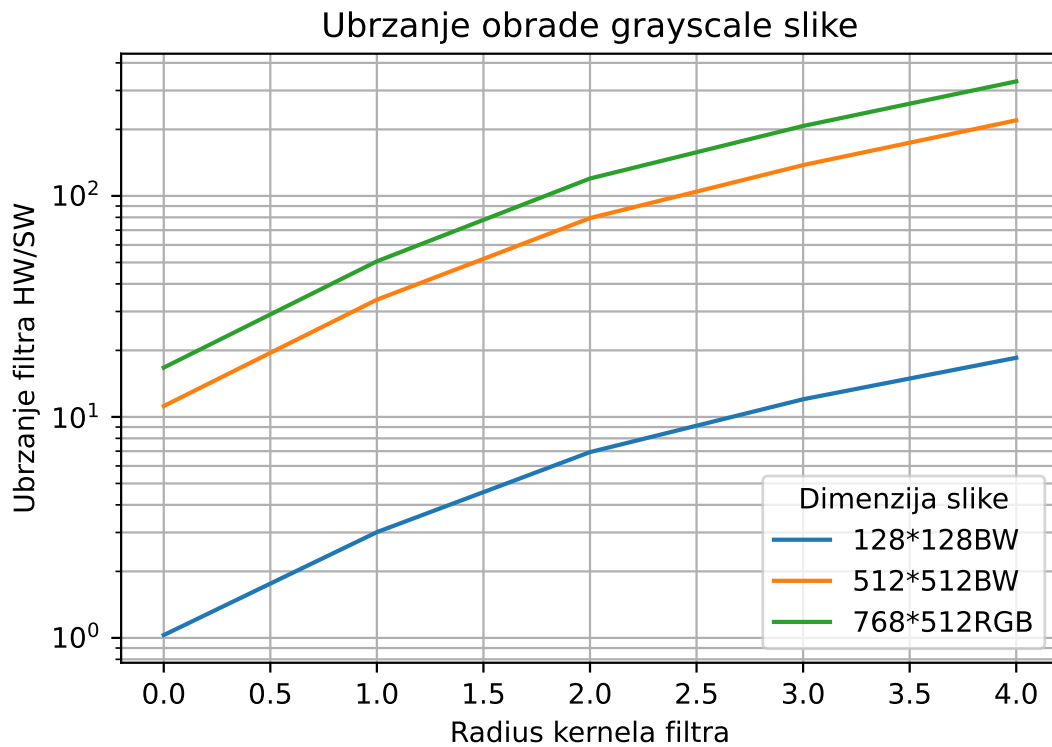
Tačnost akceleratora proverava se pozivom funkcije `CheckData`, koja upoređuje izlazne matrikse HW modela sa softverskom referencom generisanom pomoću `imageFilterSW`.

Vreme izvršavanja računa se po formuli:

$$T_{exec-us} = \frac{\text{TimerCycles} \times 10^6}{f_{AXI_TIMER}} \quad (1)$$

Ubrzanje (*Speedup*) iznosi:

$$\text{Speedup} = \frac{T_{SW}}{T_{HW}} \quad (2)$$



5.1 Primeri slika rezultata

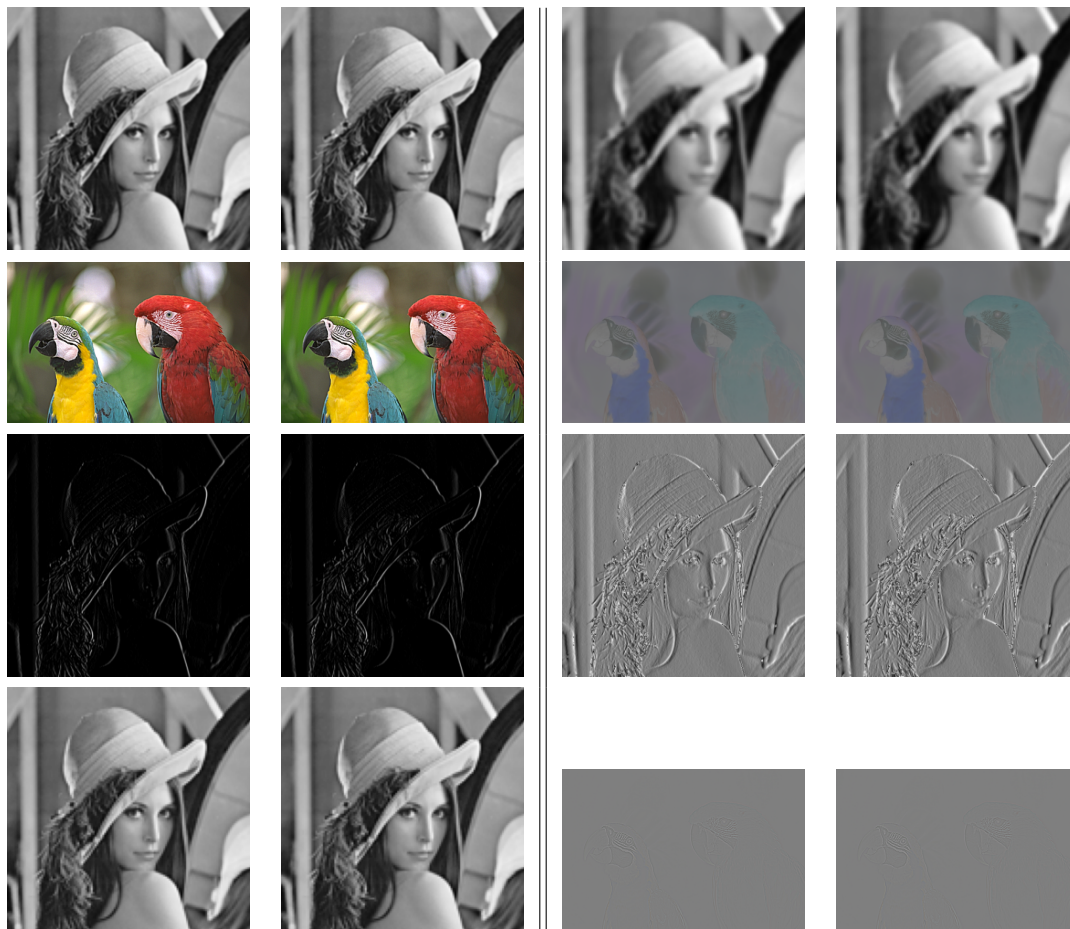


Tabela 2: Primeri slika(SW levo, HW desno),
redom box, gauss, box_sharpen, LoG r=0, Sobel, Sobel,Box r=1,LoG r=4

6 Zaključak

Uspješno je realizovan i verifikovan HW/SW sistem za ubrzano filtriranje slika na Zynq-7000 SoC. Upotrebom akceleratora u FPGA logici i AXI DMA prenosa ostvareno je značajno ubrzanje u odnosu na procesorsko izvršavanje, uz zadržavanje fleksibilnosti pri konfiguraciji filtara u realnom vremenu.